Práctica guiada: Gestor de Productos con Componentes, Data Binding y Pipes.

PASO 1: Crear el proyecto Angular

```
C:\>ng new prod-angular
✓ Which stylesheet format would you like to use? CSS             [ https://developer.mozilla.org/docs/Web/CSS
          ]
✓ Do you want to enable Server-Side Rendering (SSR) and Static Site Generation (SSG/Prerendering)? Yes
✓ Do you want to create a 'zoneless' application without zone.js? No
✓ Which AI tools do you want to configure with Angular best practices? https://angular.dev/ai/develop-with-ai None,
GitHub Copilot [ https://code.visualstudio.com/docs/copilot/copilot-customization ]
```

PASO 2: Instalar Bootstrap en el proyecto

```
C:\>cd prod-angular

C:\prod-angular>npm install bootstrap
```

En el fichero **angular.json** dentro de "styles" escribiremos

```
"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/styles.css"
```

Con esto ya puedes usar clases de Bootstrap en cualquier componente.

PASO 3: Crear el componente

Creamos el componente:

```
C:\prod-angular>ng generate component producto

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

   No
Global setting: enabled
Local setting: disabled
Effective status: disabled
CREATE src/app/producto/producto.spec.ts (565 bytes)
CREATE src/app/producto/producto.ts (205 bytes)
CREATE src/app/producto/producto.css (0 bytes)
CREATE src/app/producto/producto.html (24 bytes)

C:\prod-angular>ng generate component producto

Would you like to share pseudonymous usage data about this project with the Angular Team
at Google under Google's Privacy Policy at https://policies.google.com/privacy. For more
details and how to change this setting, see https://angular.dev/cli/analytics.

   (y/N) n
```

Deseas compartir **datos de uso seudónimos** con el equipo de Angular en Google.

PASO 4: Crear el modelo de datos

src/app/producto/producto.model.ts

```
export interface Producto {
  codigo: string;
  nombre: string;
  descripcion: string;
  imagen: string;     // ruta dentro de assets
  precio: number;
  stock: number;
}
```

PASO 5: Añadir imágenes en /assets

1835-wd-blue-sa510-25-2tb-ssd-sata-3	15/01/2026 11:47	Archivo JPG	55 KB
lenovothinkPadE14	11/12/2025 13:25	Archivo JPG	5 KB
LGUltraGear27	11/12/2025 13:25	Archivo JPG	7 KB
LogitechMXKeys	11/12/2025 13:26	Archivo JPG	5 KB

Modificar el fichero angular.json para añadir la ubicación de las imágenes

```
"assets": [
  {
    "glob": "**/*",
    "input": "src/assets",
    "output":"/assets"
  }
],
"styles": [
  "node_modules/bootstrap/dist/css/bootstrap.min.css",
  "src/styles.css"
```

PASO 6: Añadir datos usando signals

src/app/producto/producto.component.ts

```
import { Component, signal, computed } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';
import { StockPipe } from '../stock.pipe';
import { Producto } from './producto.model';

@Component({
  selector: 'app-producto',
  standalone: true,
  imports: [
    CommonModule,
    FormsModule,
    StockPipe
  ],
  templateUrl: './producto.component.html',
  styleUrls: ['./producto.component.css']
})
export class ProductoComponent {

  // Signal para el texto del buscador
  search = signal('');

  // Valor simple para el stock mínimo
  minStock = 0;

  // Signal con la lista de productos
  productos = signal<Producto[]>([
    {
      codigo: 'P001',
      nombre: 'Teclado mecánico',
      descripcion: 'Teclado RGB con switches azules.',
      imagen: '/assets/LogitechMXKeys.jpg',
      precio: 59.99,
      stock: 12
    },
    {
      codigo: 'P002',
      nombre: 'Ratón inalámbrico',
      descripcion: 'Ratón ergonómico con batería recargable.',
      imagen: '/assets/raton.jpg',
      precio: 29.99,
      stock: 5
    }
```

```
  ]);

  // Signal computado: se recalcula automáticamente cuando cambian
search() o productos()
  productosFiltrados = computed(() =>
    this.productos().filter(p =>
      p.nombre.toLowerCase().includes(this.search().toLowerCase())
    )
  );
}
```

PASO 7: Vista con Bootstrap + Data Binding

Aquí aplicamos:

- **Interpolación**
- **Property binding**
- **Event binding**
- **Two-way binding con ngModel**
- **Clases de Bootstrap**

src/app/producto/producto.component.html

```
<h2 class="mb-4">Productos</h2>

<!-- Buscador -->
<div class="row mb-3">
  <div class="col-md-6">
    <input
      type="text"
      class="form-control"
      placeholder="Buscar producto..."
      [ngModel]="search()"
      (ngModelChange)="search.set($event)"
    />
  </div>

  <div class="col-md-3">
    <input
      type="number"
      class="form-control"
      min="0"
      placeholder="Stock mínimo"
      [ngModel]="minStock"
      (ngModelChange)="minStock = $event"
    />
  </div>
```

```
</div>

<!-- Cards de productos -->
<div class="row g-4">
  <div
    class="col-md-4"
    *ngFor="let p of productosFiltrados() | stock:minStock"
  >
    <div class="card shadow-sm">
      <img
        [src]="p.imagen"
        class="card-img-top"
        [alt]="p.nombre"
      />

      <div class="card-body">
        <h5 class="card-title">{{ p.nombre }}</h5>
        <p class="card-text">{{ p.descripcion }}</p>

        <p class="fw-bold text-primary">
          {{ p.precio | currency:'EUR' }}
        </p>

        <span
          class="badge bg-success"
          *ngIf="p.stock > 0"
        >
          Stock: {{ p.stock }}
        </span>

        <span
          class="badge bg-danger"
          *ngIf="p.stock === 0"
        >
          Sin stock
        </span>
      </div>
    </div>
  </div>
</div>

<!-- Sin resultados -->
<div
  class="alert alert-warning mt-4"
  *ngIf="(productosFiltrados() | stock:minStock).length === 0"
>
  No hay productos que coincidan con los filtros.
</div>
```

PASO 8: Estilos del componente (encapsulación Emulated)

src/app/producto/producto.component.css

```
.card img {
  height: 200px;
  object-fit: cover;
}
```

PASO 9: Crear un pipe personalizado

Creamos uno para filtrar por stock mínimo:

```
C:\prod-angular\src\app>ng generate pipe stock
CREATE src/app/stock-pipe.spec.ts (191 bytes)
CREATE src/app/stock-pipe.ts (227 bytes)
```

src/app/stock.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'stock',
  standalone: true
})
export class StockPipe implements PipeTransform {
  transform(productos: any[], minimo: number): any[] {
    return productos.filter(p => p.stock >= minimo);
  }
}
```

src/app/producto/producto.component.html

```
<h2 class="mb-4">Productos</h2>

<!-- Buscador -->
<div class="row mb-3">
  <div class="col-md-6">
    <input
      type="text"
      class="form-control"
      placeholder="Buscar producto..."
      [ngModel]="search()"
      (ngModelChange)="search.set($event)"
    />
```

```
  </div>

  <div class="col-md-3">
    <input
      type="number"
      class="form-control"
      min="0"
      placeholder="Stock mínimo"
      [ngModel]="minStock"
      (ngModelChange)="minStock = $event"
    />
  </div>
</div>

<!-- Cards de productos -->
<div class="row g-4">
  <div class="col-md-4" *ngFor="let p of productosFiltrados() |
stock:minStock">
    <div class="card shadow-sm">
      <img [src]="p.imagen" class="card-img-top" alt="{{ p.nombre
}}">

      <div class="card-body">
        <h5 class="card-title">{{ p.nombre }}</h5>
        <p class="card-text">{{ p.descripcion }}</p>

        <p class="fw-bold text-primary">
          {{ p.precio | currency:'EUR' }}
        </p>

        <span class="badge bg-success" *ngIf="p.stock > 0">Stock: {{
p.stock }}</span>
        <span class="badge bg-danger" *ngIf="p.stock === 0">Sin
stock</span>
      </div>
    </div>
  </div>
</div>

<!-- Sin resultados -->
<div class="alert alert-warning mt-4" *ngIf="(productosFiltrados() |
stock:minStock).length === 0">
  No hay productos que coincidan con los filtros.
</div>
```

PASO 10: Modificación aplicación

/src/app/app.ts

```
import { Component, signal } from '@angular/core';
import { RouterOutlet } from '@angular/router';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [RouterOutlet],
  templateUrl: './app.html',
  styleUrls: ['./app.css']
})
export class App {
  title = signal('Mi Tienda Informática');
}
```

/src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { ProductoComponent } from
'./producto/producto.component';

export const routes: Routes = [
  { path: '', component: ProductoComponent },
  { path: 'productos', component: ProductoComponent }
];
```

/src/app/app.html

```
<nav class="navbar navbar-expand-lg navbar-dark bg-dark">
  <div class="container-fluid">
    <a class="navbar-brand" href="#">{{ title() }}</a>

    <div class="navbar-nav">
      <a class="nav-link" routerLink="/">Inicio</a>
      <a class="nav-link" routerLink="/productos">Productos</a>
    </div>
  </div>
</nav>

<div class="container mt-4">
  <router-outlet></router-outlet>
</div>
```